# The Law of Graphic Equalisation

Short Form

Matthew Parslow
Independent Researcher

**Draft — draft-2026-08-29.3**
29 August 2026 • short form

**SHORT FORM.** This is a generated abridgement of *The Law of Graphic Equalisation* (Theory). It states the substrate and its consequences without demonstrating the removals that establish how little is required; the long paper carries those, and is the reference for any claim here that a reader wants shown rather than asserted. Every definition and proposition below is pulled from the long paper at build time, so the two cannot drift.

**Abstract**

One minimal structure is proposed as underlying phenomena usually studied apart. This short form gives the structure, the frame machinery through which anything reaches it, the single prohibition it carries, and the commitments on which it fails. It does not rehearse the removals: those are the bulk of the long paper, are what establishes how little is required, and are worth reading by anyone who doubts a particular one.

## 1 The substrate

The substrate is structure, the transformations operative upon it, and a grammar of which relations may exist. Only the first is required for there to be anything; the other two are what a world in which anything *happens* costs. Nothing beyond the three is primitive, and anything else exists where a graph happens to represent it.

**Definition 1.1** (Graph)**.** *A graph is*

$$\boxed{G = (g, r, \Gamma),}$$

*where $g$ is represented state, $r$ is operative transformation structure, and $\Gamma$ is the admissibility grammar governing graph-originated transformations.*

**Definition 1.2** (Grounding)**.** *A* grounding *is a particular assignment of rule content and initial conditions to the substrate — one $(g, r, \Gamma)$ with its laws and its starting configuration fixed — together with the domain it is taken to model. The substrate is what every grounding shares; a grounding is what any one of them adds. Claims made about the substrate are accordingly claims about all groundings, and are correspondingly weak; claims made within a grounding carry that grounding's contingent rule content and are where this account's falsifiable commitments mostly sit.*

**Definition 1.3** (Admissible continuation)**.** *A continuation $\delta$ belongs to $\mathcal{P}(G_t)$ iff it is expressible in the current graph vocabulary, satisfies the relevant antecedent conditions, and is admissible under the operative grammar for the transformation class in question.*

**Definition 1.4** (Collapse)**.** *Collapse is the frame-relative passage from a pre-realisation possibility structure to realised continuation:*

$$\boxed{\mathcal{C}_t : \mathcal{P}(G_t) \rightsquigarrow G_{t+1}.}$$

*It is not identified with deliberative choice.*

The Law is that a graph's admissible continuations are fixed by its present structure, and that realisation selects one of them, which then becomes the antecedent for what follows.

**Definition 1.5** (Residual)**.** *The residual is*

$$\boxed{\epsilon = \delta^R \ominus \delta^P,}$$

*where $\ominus$ is the grounding's difference relation preserving relevant discrepancy.*

*Remark* 1.6 (Only $g$ is required; the triple is what a *running* world costs). The triple is not the minimum and this paper has previously implied that it was. Strip $r$ and $\Gamma$ and something remains: structure, with relations among its parts. Nothing *happens* in it — no transformation, so no collapse, no residual, and nothing it individuates — but it has not thereby become nothing, and it has not stopped being individuable by something else. What $r$ and $\Gamma$ buy is *occurrence*, and everything this account is about — change, collapse, observation, cost, experience — requires them. The triple is the price of a world in which anything happens rather than the price of there being anything, and Remark 5.2's "only that rules run is compulsory" should be read in that narrower sense.

A distinction is doing the work there. Being a frame is being individuated (Definition 2.1), and individuation is performed by the container, not by the thing: a boundary belongs to the frame that draws it. So a structure with no rules of its own is still a frame — something else instantiated its boundary — and what it lacks is not framehood but the capacity to individuate anything itself. The earlier claim that stripping $r$ leaves "no frame" was wrong, and it would have invalidated the companion result that reasons from a rock's being a frame to the universe's being one.

## 2 Frames

A frame is how anything is reached at all: there is no view from nowhere, so every claim is a claim made in some frame, and the frame is not a container holding contents but the relation that demarcates them.

**Definition 2.1** (Frame)**.** *A* frame *is an instantiated boundary relation — the edge $\mathcal{B}_{W:G}$ itself, and not the subgraph $G$ it demarcates. The subgraph is the relatum; the frame is the relation. To individuate is to instantiate such an edge, which is why individuation is per frame rather than a fact about the graph.*

**Corollary 2.2** (Frame and dimension are one edge type in two roles)**.** *A dimension is the same construction read from the other end. The boundary $\mathcal{B}_{W:G}$ is a dimension of $W$ — one of the properties along which $W$ separates what it contains — and each dimension $W$ instantiates is a frame to the values it separates. So* frame *and* dimension *name two roles one edge type plays relative to another edge, not two kinds of thing, and the asymmetry between them is positional in the same way that the asymmetry between a parent and a child is.*

**Proposition 2.3** (One interface, two gates)**.** *A situated graph meets its containing frame through a single relation, and what an observation from that frame delivers is not distinct in kind from what the containing frame imposes: to a subgraph, being observed* is *being deformed by its*

*container. That one relation is gated twice, by the same kind of admissibility grammar on its two faces — the* aperture grammar $\Gamma_a$ *on the input and the* change grammar $\Gamma_c$ *on the output:*

$$\boxed{\underbrace{\textit{aperture}}_{\Gamma_a} \longrightarrow \underbrace{\textit{rules}}_{r} \longrightarrow \underbrace{\textit{change}}_{\Gamma_c}.}$$

$\Gamma_a$ *limits, by grain, which of the container's deformation enters the rules;* $\Gamma_c$ *determines which of the rules' realised deltas cohere into the container and which remain as residual (Proposition (long form)). Both are faces of the frame's one grammar* $\Gamma$*, and instantiate for a specific container* $W$ *as* $\Gamma_{W_a}$ *and* $\Gamma_{W_c}$*; it is* $\Gamma_{W_c}$*, the change face, that the residual proposition uses. Execution runs between the two.*

**Definition 2.4** (Aperture)**.** *A frame's* aperture *is its input gate* $\Gamma_a$ *together with what that gate fixes: which dimensions it can separate along at all, how finely it separates within each (its resolution, per dimension, for which this paper also uses* grain*), and how far along each it reaches. The three are properties of one edge — existence, grain, and extent — and are named separately because they vary independently, not because they are different kinds of limit. What falls outside an aperture on any of the three is* absent *to that frame rather than held-but-unresolved.*

**Definition 2.5** (Change grammar)**.** *A frame's* change grammar $\Gamma_c$ *is its output gate: it determines which of the realised deltas its rules produce cohere into the containing frame and which do not. What it declines has entered the frame and been operated on, so it is* residual *(Definition 1.5) rather than absent, and it persists in place rather than being discarded.* $\Gamma_a$ *and* $\Gamma_c$ *are the two faces of one grammar* $\Gamma$ *on one boundary edge, and the asymmetry between them is which direction they gate, not what kind of thing they are.*

**Definition 2.6** (States of a dimension)**.** *For a frame* $\mathcal{F}$*, a dimension is in exactly one of four states:*

$$\boxed{\text{NULL} \neq \text{undetermined} \neq \text{undisclosed} \neq \text{resolved}.}$$

NULL*: the dimension is not instantiated; there is nothing to have a value.* Undetermined*: the dimension is instantiated and* no value has been realised *— collapse has not occurred.* Undisclosed*: a value has been realised but is not held by this frame.* Resolved*: a realised value, held.*

# 3 Economics

Cost attaches to writes rather than to holding, and what a write costs is set by whatever runs the rules rather than by any universal.

**Proposition 3.1** (Cost characteristically reduces to the write)**.** *In the substrate as characterised here, the boundary relation itself is free; the standing structure persisting is free; the operation that costs is a delta that writes over that structure, and* $r$ *is what writes. Free here means information-preserving rather than absent — evaluating a rule is itself a write, and is unpaid because it destroys nothing (Remark 3.3). To that extent*

$$\boxed{\textit{cost is paid by rules.}}$$

*This is a characterisation of the substrate as specified, not a necessity of graphhood, and the writeability status it describes is not required to be uniform (Remark (long form)).*

**Definition 3.2** (Well-formedness)**.** *A graph is* well-formed *to the degree that it pays for nothing that does no work for it. Three costs bear on that and they behave differently, so the notion is graded along each rather than binary.*

Writes *are paid when made (Proposition 3.1), which prices doing.* Retention *is nearly free per unit, which is why an unused distinction is not immediately pruned.* Search *is paid on every operation over what is held and scales with how much is held, so inert structure is cheap to keep and expensive to keep in quantity.*

*The fourth is the worst and is not a variant of the third. A write may land on structure that cannot affect anything — a modification to a branch nothing reaches. It is paid in full: decided, made, checked, risked. It returns nothing. And unlike a search, which is expensive and then yields, this failure is* silent*: the graph has spent, has no effect to show for it, and no signal that the effect is missing. What it acquires instead is a false model of itself, in which something was changed that was not — residual it cannot see, produced by its own inert contents, and the share of writes exposed to it grows with how much is held.*

*So a graph does not become ill-formed by carrying something useless. It becomes ill-formed by carrying enough that finding the useful parts costs, and enough that its changes start landing where nothing happens.*

*The third cost is where aperture enters, and it is why the same contents are well-formed for one graph and not another. What search costs depends on how finely the graph can discriminate within what it holds: with structure that separates the relevant from the rest, the cost grows slowly with size; without it, every operation is over everything. That discriminating structure is itself held and itself earns its keep, so a well-formed graph is one whose aperture over its own contents keeps pace with their growth.*

*Removal is therefore not free either, being a write. Pruning happens when accumulated cost exceeds the one-off cost of removal, and the fourth cost is what usually triggers it: search cost is diffuse and tolerated for a long time, while a change that demonstrably did nothing is a discrete and memorable event. Systems clean up after being burned rather than after slowly noticing. Well-formedness is graded rather than binary; it is relative to* what work *the graph is doing, so a graph well-formed for one purpose is not thereby well-formed for another; and it is not enforced by the substrate. An ill-formed graph runs. It pays more to.*

*Remark* 3.3 (Evaluation is a write, and that is why it is free)*.* The economics above can be stated in a way that does not survive contact with the machinery, and the objection is worth answering rather than rewording. If running a rule that does not fire is a read, and reads are free, where does the check go, and how does the substrate know to run the next one?

Computing answers it without any appeal to physics. `read()` is an operation, and it returns data; the returned datum has to be somewhere to be used, and in a substrate that is graph all the way down there is no return channel outside the structure, so the value materialises as structure. A read therefore writes by being a write, and the write is the return. An operation that returns nothing is not a read of anything, which is why the free case exists and is exactly the empty one: a read whose return is never used has read nothing, and a compiler will delete it. Real machines make the same point expensively — a read leaves traces in caches, predictors and translation state, and those traces are measurable, which is why timing attacks recover secrets from a victim whose only secret-dependent operation is a read (long form) and why an entire class of speculative attacks was available to designs built on the assumption that rolled-back speculation leaves no trace (long form), where the read is the vehicle rather than the premise.

Evaluation is a write in that plain sense: a match outcome, a position in an ordering, and the fact of having tried are returned values, and returned values are structure. What does not follow is the price. A rule's cost is set by whatever supplies its primitives and is inherited along provision (Section (long form)), and Landauer's bound (Remark (long form)) is a floor in one substrate rather than the price in any. This is also why continuation presents as free from inside: a well-formed runner does not expose itself (Proposition 5.4), so its charge is below the frame's aperture and is not an in-frame event at all. In-frame nothing happened; in the running substrate something was paid; both hold, of different frames.

Two things follow for the rest of these papers. Reversibility is a structural fact about a

map and cost is a fact about what supplies a primitive, and conflating them is what the earlier phrasing did — separating them is what lets the physics companion recover unitarity, which needs only the first, since a no-write was never the right description of an evolution that plainly changes the state and is reversible because it is injective. And the residual named in Remark (long form) attaches here: information-preserving evaluation accumulates the history it preserves, so a frame running a long time without collapsing has deferred a cost into retention rather than escaping one. What is free is not the running, only the not-yet-forgetting — and free only relative to a frame that cannot see the bill.

**Proposition 3.4** (Primitives leak, and complete provenance is still unrecoverable)**.** *Leaking is ordinary rather than exceptional. A primitive's timing discloses the structure beneath it, which is why measuring how long a read takes recovers what a cache holds (long form); an arithmetic that overflows at a particular magnitude discloses a word size; a rounding discloses a representation; and a performance cliff at a particular size discloses a page. Most abstractions leak, and this one is not special.*

*How a leak reaches a frame at all is worth stating, since what falls outside an aperture is* absent *(Definition 2.4) and an absent thing cannot disclose anything. It does not arrive by perception. What arrives is an ordinary in-frame observable on a dimension the frame already instantiates — a duration in its own units, a value at a boundary — and what the frame does with it is* infer*. The cache is not perceived; the timings are, and the cache is concluded from them.*

*What does not follow is that provenance can be recovered entire. Every act of recovery is itself performed by primitives, so a graph inferring what lies beneath one operator does so using others whose own provenance is not thereby in view, and there is no position from which the whole supply is inspected at once. The result is the standing shape:* partial always, complete never *— leaks give a graph real and sometimes decisive knowledge of what supplies it, and never the closure of that question.*

# 4 Time

Time is not a substrate dimension. It is inferred by a frame from its own closure cycles, which is why a frame that runs no rules has none.

**Proposition 4.1** (Time is inferred, not observed)**.** *No collapse presents a temporal coordinate. Succession is a relation* between *realised states, and a relation between states is not a datum of any one of them. A frame therefore acquires a time dimension only by* inferring *it from the relations among what it holds. Consequently*

$$\boxed{\textit{a frame observes time} \implies \textit{that frame is capable of inference.}}$$

**Proposition 4.2** (The unit of inferred time is a cycle, and requires closure)**.** *A frame does not count its individual changes — those are below its aperture — but its own* cycles*, each one traversal of a closed rule-loop. The unit of inferred time is therefore a cycle, not a change, and a cycle has no fixed length: each does a variable amount of change, so inferred time is non-uniform in underlying change (a nonlinear grounding, which Discrete Sufficiency already permits) and a fuller cycle reads as a longer moment (subjective dilation). A cycle exists only for a graph with* closure *— a graph without it runs to fixed point once and stops, one pass and no repeated tick — so having time requires closure as well as inference (Proposition 4.1): a one-shot computation, lacking a loop to iterate, has no time, and a frame's experienced "now" is one cycle of its own loop.*

**Definition 4.3** (The present, structurally)**.** *Within a frame, the present is the collapse boundary at which every instantiated* undetermined *dimension relevant to that collapse has been determined:*

$$\boxed{U_{\mathcal{F}} = 0.}$$

*$U_{\mathcal{F}}$ counts undetermined dimensions only. NULL dimensions are excluded — they are not variables.* ***Undisclosed dimensions are also excluded****: they are already determined, and a frame's failure to hold them is an epistemic deficit rather than an unresolved collapse. A frame with outstanding undisclosed structure is still at its own present.*

# 5 Primitives, and the single prohibition

**Definition 5.1** (Primitives)**.** *A graph's* primitives *are the operations it invokes without performing: the operator set its rules bottom out in. Every level has them, and having them is what being a level consists of — a graph with no primitives would have to perform everything it invokes, which is the regress this substrate does not contain. That is the whole of the definition, and in particular it says nothing about what a graph can learn of where they come from.*

*Two different things can be meant by provenance being hidden, and running them together inverts the relation between them.*

Deliberate exposure — *state maintained so that what runs on it can read it, as reflection does — is structure doing work for something other than the frame holding it, and is therefore excluded by well-formedness (Definition 3.2) in the ordinary way: it costs, and the frame supplying it gets nothing.*

Leakage *is the opposite case and is not excluded at all. A cache earns its retention many times over and leaks precisely* because *it does work; an arithmetic fast enough to overflow at a word boundary discloses the boundary by being fast; a page-sized performance cliff exists because the page size is doing something. Every leak in Proposition 3.4 is a consequence of efficient design rather than of poor design, so well-formedness does not prevent leakage — it* produces *it, and a structure that leaked nothing would have given up the optimisations that distinguish it from a uniform one, which is what constant-time implementations do and what they pay for.*

*Remark* 5.2 (What is required and what is optimisation). Only that rules run is compulsory for anything to *occur* — structure alone is what is required for there to be anything at all (Remark 1.6). Applying a delta is itself a rule, not a primitive operation beneath the rules, so the specific dynamics — inertia, the merge behaviour, closure — are contingent rule content, and a different $r$ or $\Gamma$ is a different physics rather than a violated law (this is why the initial and boundary conditions of a grounding are supplied separately from its laws; Section (long form)). The two gates of Proposition 2.3, and even the retained standing structure itself, are *optimisations* on bare rule-running: the aperture grammar $\Gamma_a$ bounds the input, the change grammar $\Gamma_c$ sorts the realised output into what coheres and what is residual, and the retained structure buys free continuation. A substrate whose grammar admits everything still runs — effectively ungated, without momentum, wastefully; it is not a substrate *lacking* $\Gamma$, since Definition 1.3 defines the possibility structure through the grammar and a collapse needs something to select from — so they are what an efficient graph accretes under a budget, not constituents of graphhood. The retained structure specifically is what distinguishes a self-sustaining loop from a purely reactive one, since coasting is the cached structure.

**Proposition 5.3** (Primitives are supplied, not intrinsic)**.** *A graph is data — nodes, relations, and rules — and the operations its rules bottom out in are supplied to it rather than contained in it. The same graph may therefore be run against different supplies of the same operator set, and nothing supplying an operator need be the substrate the graph describes or is "in." Where the work is done is a fact about that supply and not about the graph.*

**Proposition 5.4** (A well-formed runner does not expose itself, and no level is self-transparent)**.** *The asynchronous, locally synchronous structure a frame observes (Remark (long form)) is the graph's* internal *view and does not fix its runner's actual mode. A hidden central runner may execute a graph serially, one change at a time, while the graph — lacking any access to the runner's clock — appears asynchronous and concurrent to itself. More generally, where a frame has no boundary to the substrate that runs it, its runner lies beyond the frame's epistemic boundary in exactly the sense of Section (long form).*

*Two claims have to be separated here, and an earlier statement of this proposition ran them together by calling the runner unseeable rather than unseen. That a runner is hidden is* contingent*. Nothing forbids a runner from exposing its own state to what it runs, and computational reflection does exactly that, as this paper acknowledges when placing itself against that literature (Section (long form)); a tower in which each level is available to the one above is a built artefact, not a thought experiment. A runner that does not expose itself is the* well-formed *case, and for the reasons well-formedness always turns on: exposed runner state is structure maintained for work the frame does not do, and each exposure needs a runner of its own to run the reading of it.*

*What is not contingent is weaker and survives reflection:* no level is self-transparent*. Reading is an operation whose product lands one level up (Remark 3.3), so a level cannot hold the reading of itself while being the level that is running. Reflection makes the levels explicit and pays for each; it does not terminate in a level transparent to itself. A frame may therefore be given sight of what runs it, and cannot be given sight of its own running. Since the rules are graph and a frame can read and rewrite them (Definition 2.1), while its own running is not graph it holds, the asymmetry is exact: a frame is transparent to itself in precisely one respect and opaque in precisely the other.* It can know what it is and cannot observe its being it. *Self-modification is available; self-observation at the level of the modifying is not.*

**Proposition 5.5** (A primitive is another graph's running, exposed)**.** *Nothing supplying a primitive is a substance apart from graph data. An operator is the observable running of some other graph, which has primitives of its own, supplied in the same way. A graph's "primitives" are therefore the observable running of other graphs, exposed as an operator set, with no primitive runner at the base. A single chain — code over machine code over microcode over silicon over electromagnetism over quantum physics — is one path through that provision, and a common one, but not its shape: a process takes arithmetic from its processor, file operations from a kernel that relays them to a device, and remote calls from a machine that is not beneath it in any sense. Fan-out and relay are ordinary, and the tower is the degenerate case in which provision happens to be a chain. The regress bottoms not at a floor but at the epistemic frontier of Section (long form): below the lowest measured level is the unclaimed pre-relational substrate.*

The account forbids one thing: a frame cannot get outside itself. It cannot be transparent to itself at the level it is running at, cannot measure its own throughput, and cannot hold a view from nowhere. That prohibition has a formal analogue — a language of sufficient strength cannot define its own truth predicate, and must be given it from a metalanguage — and the analogue is proved, though the mapping from frames to formal systems is argued rather than proved. What is forbidden is complete self-capture and not partial self-modelling, which is why reflective towers work and a frame can know a great deal about itself.

## 6 Carvings

Since the rules draw the boundary and the rules are contingent, no carving is privileged. That does not make the notion vacuous: an ill-formed cut costs more writes, so a frame under a budget tends to be running cheaper cuts, and the cuts that look like joints are the cheap ones rather than the real ones.

**Proposition 6.1** (Convergence is observed, not enforced)**.** *Two frames doing the same work converge on similar cuts as each becomes better formed, because the cheap cut for a given task is nearly unique. This is a tendency of cost and not the discovery of a privileged boundary, and the two are distinguishable in principle: a privileged boundary would* enforce *convergence, and enforcement would extend to the hard cases; cost-shaped cuts converge where the work is shared and come apart where it is not. The step from shared work to similar cuts assumes the cheap cut for a task is nearly unique, which is an empirical posit and not a result of this account; it is labelled again where it is used (Proposition 6.2).*

**Proposition 6.2** (Convergence under compression)**.** *A structure operating under a constraint, and under pressure to spend nothing on what does no work, approaches the arrangements that constraint forces — asymptotically, and only over the region the constraint touches. Two such structures therefore come to resemble each other without either having access to the other, because both are downstream of the same forcing rather than converging on a target.*

**Hypothesis 6.3** (Located degradation)**.** *Where a classification's purpose can be specified independently and in advance of inspecting its failures, the failures concentrate near the boundary of that purpose more than a uniform distribution over the domain would produce. The claim is comparative and is not that failures occur nowhere else.*

# 7 Falsifiability

The substrate risks little by design and the groundings carry the risk. Below are the commitments attached to the material this abridgement states. They are a subset: the long paper carries the full set, including those for results not stated here, and the generator reports the difference on every build so the omission stays visible rather than becoming a claim.

1. [SHAPE] **Minimal graph sufficiency.** Fails if basic realised graph dynamics require a primitive beyond $(g, r, \Gamma)$ rather than represented or relational structure.

2. [SHAPE] **Discrete sufficiency.** Fails if a claimed observable consequence cannot be preserved by any discrete representation, including dynamically varying quantisation, but is preserved by a genuinely non-discrete primitive.

3. [SHAPE] **Antecedent-state principle.** Fails if a realised transition can require as antecedent a fact that exists only as consequence of that same transition, absent prior fixed-point structure.

4. [SHAPE] **Graph-law determinism.** Fails if two genuinely identical, causally complete antecedent configurations produce different realised successors without any additional causally operative distinction. Variation observed only from a frame that omits determining dimensions does not constitute such a falsification.

5. [DETAIL] **Located degradation.** Fails if, for classifications whose purpose is specifiable in advance of inspecting their failures, failure cases are distributed no differently near the purpose boundary than across the rest of the domain (Hypothesis 6.3).

6. [DETAIL] **Convergence is not enforced.** Fails if convergence between independently developed classifications of one domain extends to the cases where the classifications' purposes differ, which is what a privileged boundary would produce and cost-shaped convergence would not (Proposition 6.1).

7. [DETAIL] **Provenance is never closed by inference.** Not that primitives fail to leak — they do leak, and Proposition 3.4 says so. This fails if a frame can *settle* the provenance of

its own operator set from inside: if some in-frame procedure establishes, rather than merely evidences, whether its operators are self-supplied, how many distinct structures supply them, or whether a serial hidden supply underlies an apparently asynchronous frame The escape is not that further primitives were used, which is unavoidable, but that their provenance is open in the same way, so no procedure closes the question rather than relocating it.

8. [DETAIL] **Solutions are nearly unique under a binding constraint.** The posit under Propositions 6.1 and 6.2. Fails if structures optimising hard against the same binding constraint, over the same region, are exhibited arriving at arrangements that do not resemble each other — in which case constraint narrows the space without selecting within it, and both propositions weaken to that.